

<h1>Git</h1>

In this lesson we'll start your **binabox**

- Initialize a repository
- Create a branch
- Do your first git homework
- Commit & Push
- Create a pull request



Git is a software version control system

- Distributed Version Control
- Commit History & Tracking
- Undo & Rollback Changes
- Branching and Merging
- Hooks for Automation
- Distributed Collaboration



Github is a website

code hosting platform

- Git repository storage
- Social network
- Forking & Open Source Contribution
- Issue tracking
- Code review via pull requests
- Actions & CI/CD Integration
- Graphical User Interface (GUI) & Web Interface
- Team & Access Management

This is most probably the first place for your resume and portfolio



Most popular alternatives:



[GitLab.com](https://gitlab.com)

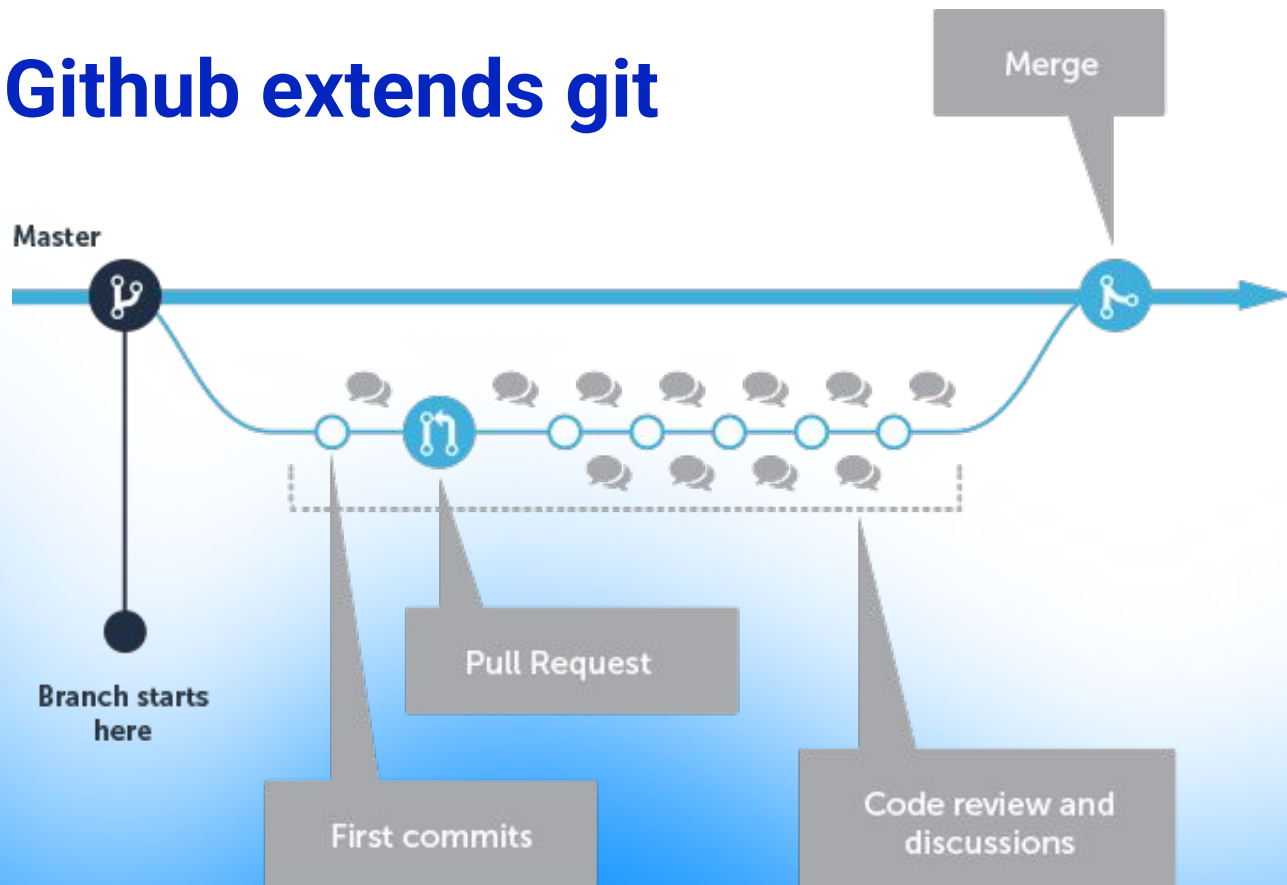


[BitBucket.org](https://bitbucket.org)

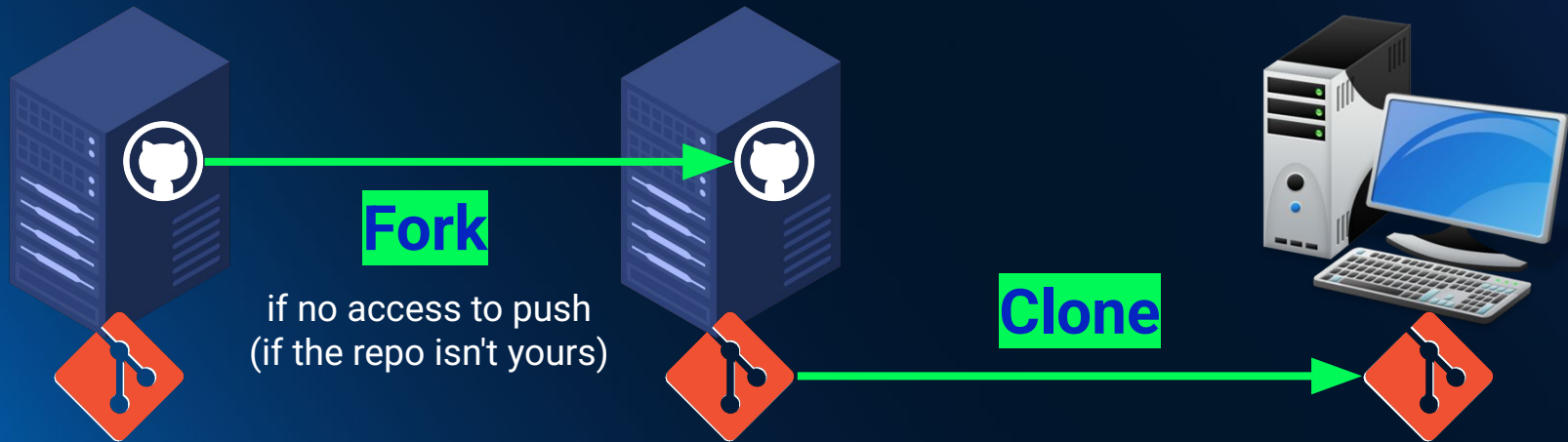


[Codeberg.org](https://codeberg.org)

Github extends git



How to get an **existing** repository



Fork is a copying a repo to place where you have enough access

Clone is copying (downloading) a repo to your local machine or into the cloud IDE

LET'S INITIALIZE

A NEW REPOSITORY: **BINABOX**



LET'S INITIALIZE

A NEW REPOSITORY: **BINABOX**

Q Type [/] to search

| >_

1

+

▼



2



New issue



New repository



Import repository



New codespace



New gist



New organization



New project



Create a new repository [Preview](#) [Switch back to classic experience](#)

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).

Required fields are marked with an asterisk (*).

bro-academy

1 General

Owner *

 fogrew

Repository name *

binabox-fogrew

Great repository names are short and memorable. How about [automatic-robot?](#)

Description

0 / 350 characters

your github username

2 Configuration

Choose visibility *

Choose who can see and commit to this repository

 Public

Start with a template

Templates pre-configure your repository with files.

No template

should be public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off ☐

[Add a README](#)

press when ready

Create repository

3 ways how to connect our repository

1 Quick setup — if you've done this kind of thing before

 Set up in Desktop or **HTTPS** **SSH** `https://github.com/alex-koshara/Binabox-Alex.git` 

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

2 ...or create a new repository on the command line

```
echo "# Binabox-Alex" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/alex-koshara/Binabox-Alex.git
git push -u origin main
```

1. Open your project in VS Code 
2. Open Terminal (Menu → View → Terminal)
3. Copy & Paste all the code from 2nd way line by line
4. Hit enter

3 ...or push an existing repository from the command line

```
git remote add origin https://github.com/alex-koshara/Binabox-Alex.git
git branch -M main
git push -u origin main
```



Ignore redundant files

What to ignore

- Generated files
- Debug logs

[Official docs](#)
[Generator](#)

One file path per line

Global: `~/.gitignore_global`

[Collection of useful global ignores](#)

System files, your editor settings

Local: `.gitignore`

Generated files, and files that you don't want to see in the repository

LET'S DO ~~HOME~~WORK

Homework

Made two iterations of pull requests

1. Git lesson (add readme)

- 1.1. Create a branch with your task about git lesson
- 1.2. Add yourself and your mentor into `README.md`
- 1.3. Commit and push these changes
- 1.4. Create a pull request and set your mentor as a reviewer
- 1.5. Wait until your mentor will merge your branch into target branch

2. Save results of HTML lesson

- 2.1. Move back to target branch
- 2.2. Create another branch with your task about html lesson
- 2.3. Add your HTML files (index.html, blog.html)
- 2.4. Commit and push
- 2.5. Create a pull request and set your mentor as a reviewer
- 2.6. Wait until your mentor will merge your branch into target branch



README.md

is a text file that introduces
and explains a project.

It contains information that is commonly required
to understand what the project is about.

Can be written in any text file format, the most
common one is **Markdown**.

Autoprefixer Cult of Martians

[PostCSS](#) plugin to parse CSS and add vendor prefixes to CSS rules using values from [Can I Use](#). It is recommended by Google and used in Twitter and Alibaba.



Write your CSS rules without vendor prefixes (in fact, forget about them entirely):

```
::placeholder {  
  color: gray;  
}  
  
.image {  
  background-image: url(image@1x.png);  
}  
  
@media (min-resolution: 2dppx) {  
  .image {  
    background-image: url(image@2x.png);  
  }  
}
```

Autoprefixer will use the data based on current browser popularity and property support to apply prefixes for you. You can try the [interactive demo](#) of Autoprefixer.

```
::-moz-placeholder {  
  color: gray;  
}  
  
::placeholder {  
  color: gray;  
}
```

Markdown



****is a simple way to format text
that looks great on any device.****

It doesn't do anything fancy like
change the font size, color, or type —
just the essentials, using keyboard
symbols you already know.

Edit

Preview

Spaces

2

```
1 # Autoprefixer [![[Cult Of Martians]][cult-img]][cult]
2
3 
6
7 [PostCSS] plugin to parse CSS and add vendor prefixes to CSS rules using values
8 from [Can I Use]. It is recommended by Google and used in Twitter and Alibaba.
9
10 Write your CSS rules without vendor prefixes (in fact, forget about them
11 entirely):
12
13 ```css
14 ::placeholder {
15     color: gray;
16 }
17
18 .image {
19     background-image: url(image@1x.png);
20 }
21 @media (min-resolution: 2dppx) {
22     .image {
23         background-image: url(image@2x.png);
24     }
25 }
26 ```
27
28 Autoprefixer will use the data based on current browser popularity and property
29 support to apply prefixes for you. You can try the [interactive demo]
30 of Autoprefixer.
31
32 ```css
33 ::-moz-placeholder {
34     color: gray;
```


Line Endings (CRLF) in **Git**

- **Why it matters?**

Ensures collaboration across OS (Windows, macOS, Linux) with different line ending characters (CRLF for Windows, LF for Unix).

- **Problem solved**

Prevents diff errors or conflicts due to mismatched line endings.

- **How to configure**

- For macOS/Linux: `git config --global core.autocrlf input`
- For Windows: `git config --global core.autocrlf true`
- Or use `.gitattributes` (e.g., `* text=auto`) for repository consistency.

[Github documentation](#)

B Academy
RO

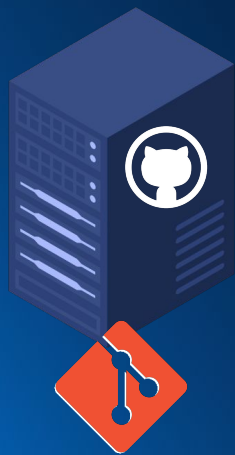


BREAK TIME

PULL

To update the **remote** changes locally





remote



Pull

Pull will synchronize
your local repo via
updating anything
from remote one



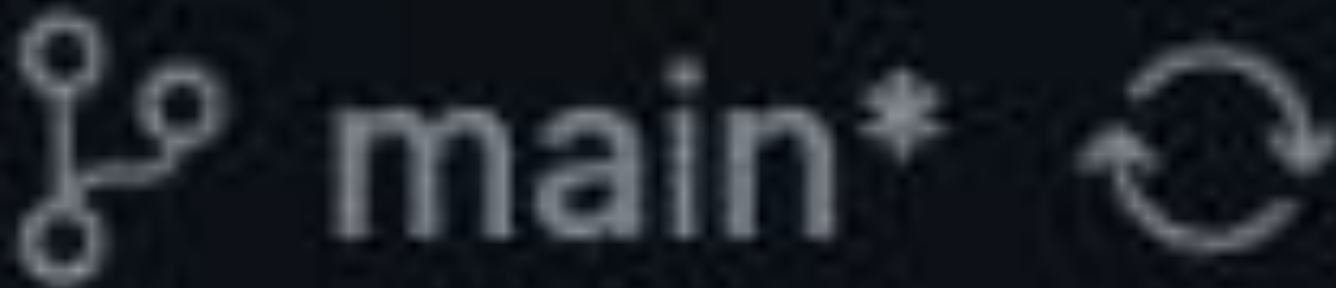
local





```
5 **[mandat1](mandats/mandat-3-git.pdf)**<br />  
6  
7 <!-- **Part 1:**<br />  
8 **[Group 1 video]  
  (https://drive.google.com/file/d/1BPzymgj0e2KRDhD2Ze9uox50AREqJHV0/view?usp=sharing)**<br />  
9 **[Group 2 video]  
  (https://drive.google.com/file/d/19PD7KW0jxcqEaYhbZ4HpEt\_cfsCbCXKD/view?usp=sharing)**<br />
```

```
5 **[mandat1](mandats/mandat-3-git.pdf)**<br />  
6  
7 <!-- **Part 1:**<br />  
8 **[Group 1 video]  
  (https://drive.google.com/file/d/1BPzymgj0e2KRDhD2Ze9uox50AREqJHV0/view?usp=sharing)**<br />  
9 **[Group 2 video]  
  (https://drive.google.com/file/d/19PD7KW0jxcqEaYhbZ4HpEt\_cfsCbCXKD/view?usp=sharing)**<br />
```



```
21 ...4. Create a pull request and set your mentor as a  
  reviewer  
22 ...5. Wait until your mentor will merge your branch into  
  target branch  
23  
24 2. **Save results of HTML lesson**  
25 ...1. Move back to target branch  
26 ...2. Create another branch with your task about html lesson  
27 ...3. Add your HTML files (index.html, blog.html)  
28 ...4. Commit and push  
29 ...5. Create a pull request and set your mentor as a  
  reviewer
```

```
21 ...4. Create a pull request  
22 ...5. Wait until your mentor  
  branch  
23  
24 2. **Save results of HTML  
25 ...1. Move back to target  
26 ...2. Create another branch  
27 ...3. Add your HTML files  
28 ...4. Commit and push  
29 ...5. Create a pull request
```

> GRAPH
> STASHES
> GITLENS

main* ↺ 0 0 Git Graph

Bro academy, last week Bro academy (1 week ago) Ln 7, Col 17 Spaces: 3

BRANCHES

HOW TO WORK **IN PARALLEL**

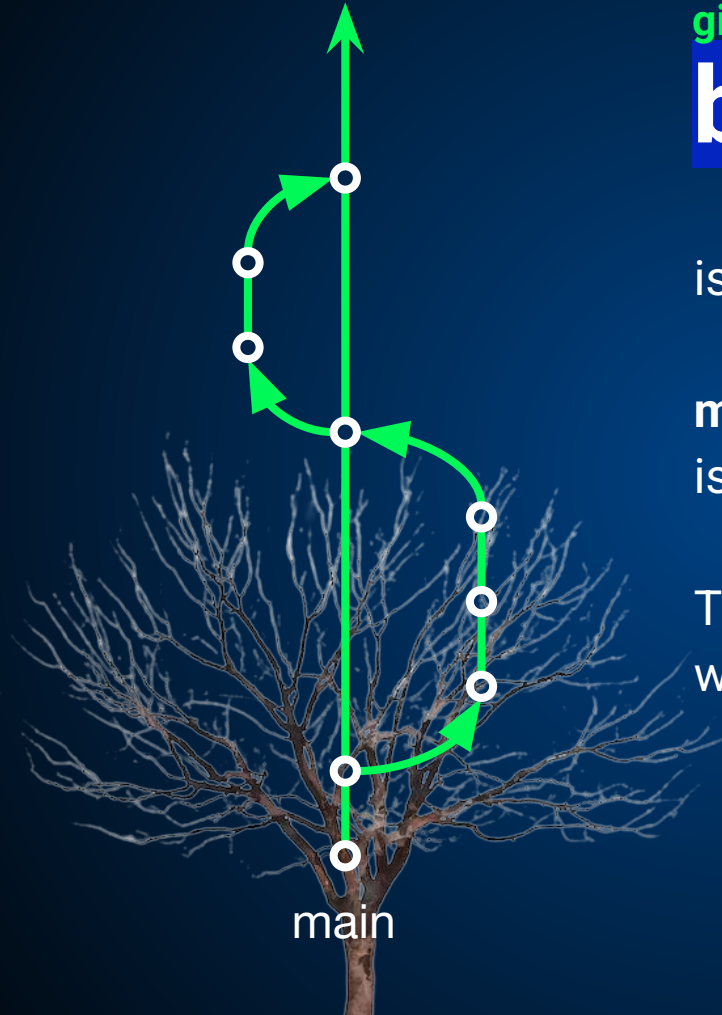


git branch

is a “parallel” development of your repo

main (somewhere still called **master**)
is the main branch of any repository.

The remaining branches can be called
whatever you like.



Good practices

- 🧑 Don't commit into target branch (main)
- 🔀 Create a branch for each task
 - Don't push changes into a previous branch
 - Don't push changes of multiple tasks into one
- ! Use team branching and branch naming rules

Branch naming

- Use **Project name** and **Task ID** from Jira

BAFC5-74-readme

- **Descriptive names**

login, navbar, fix, change

- **Alphanumeric lowercase characters separated by hyphens**

Avoid punctuation, spaces, underscores, or any special characters whenever possible

fix-login-issue

fixLoginIssue or fix_login_issue

- **Avoid Unnecessary Hyphens**

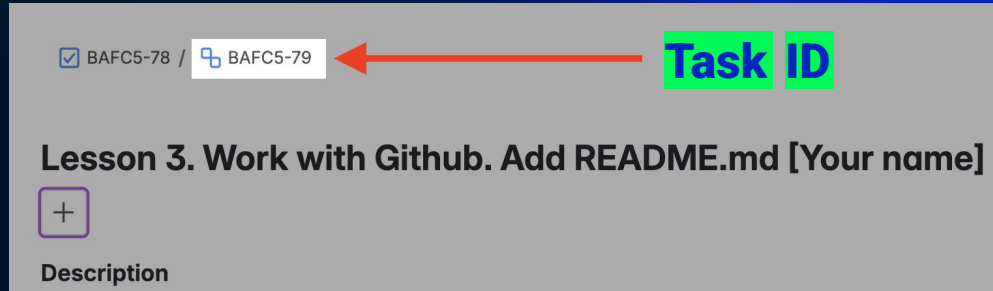
feat/new--login-

- **Short and Effective**

BAFC5-74-readme

BAFC5-74-lesson-3-readme

BAFC5-74-lesson-3-work-with-github-add-readme-md



STAGE

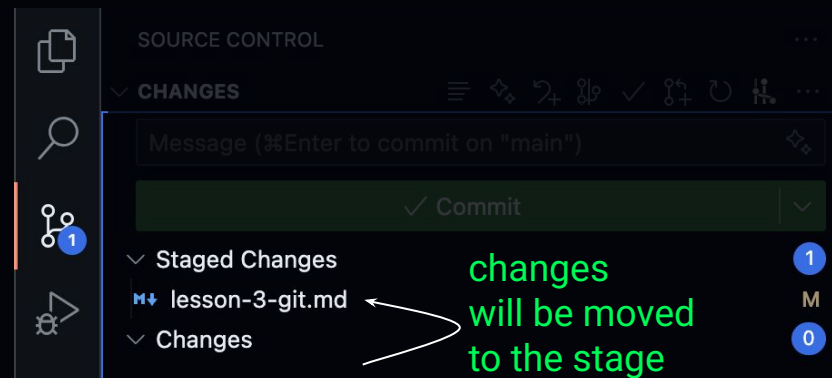
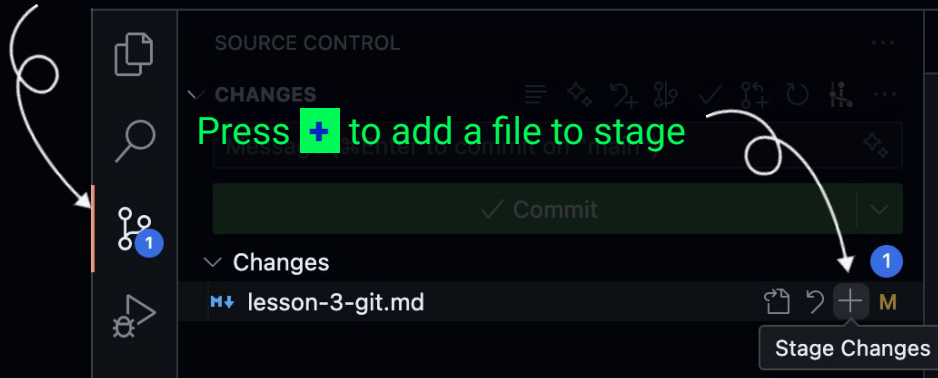
ADD CHANGES FOR A COMMIT



Staged changes

is changes that are chosen for next commit to be described in commit message

in **Source Control** tab



Staged changes

you can pick only few changes in a file

```
1 # Lesson 3. Git & Github
2
3 **[Presentation 1](presentations/presentation-3-git-1.pdf)**
4 <br />
5 **[Presentation 2](presentations/presentation-3-2-
6
7 <!-- **Part 1:**<br />
8 **[Group 1 video]
9 (https://drive.google.com/file/d/1BPzmgj0e2KRDhD2Ze9uox50AR
10 EqJHV0/view?usp=sharing)**<br />
11 **[Group 2 video]
12 (https://drive.google.com/file/d/19PD7KW0jycqEaYhbZ4HpEt_cfs
13 cbCXKD/view?usp=sharing)**<br />
14
15 **Part 2:**<br />
16 **[Group 1 video]
17 (https://drive.google.com/file/d/1zmAk6lVv0VygGwITLWrSjZtySX
18 mFwt7B/view?usp=sharing)**<br />
19 **[Group 2 video]
20 (https://drive.google.com/file/d/1aqunP8Vc9hwKfLcwr0nZzIBru6
21 AJVbkC/view?usp=sharing)**<br />
22
23 # Homework
24
25 1. **Add README.md**
26
27 1. Create a branch with your task about git lesson
28 2. Add yourself and your mentor into README.md
29 3. Commit and push these changes
30 4. Create a pull request and set your mentor as a reviewer
```

```
1 # Lesson 3. Git & Github
2
3 **[Presentation 1](presentations/presentation-3-git-1.pdf)**<br
4 />
5 **[Presentation 2](presentations/presentation-3-2-
6
7 <!-- **Part 1:**<br />
8 **[Group 1 video]
9 (https://drive.google.com/file/d/1BPzmgj0e2KRDhD2Ze9uox50AR
10 EqJHV0/view?usp=sharing)**<br />
11 **[Group 2 video]
12 (https://drive.google.com/file/d/19PD7KW0jycqEaYhbZ4HpEt_cfs
13 cbCXKD/view?usp=sharing)**<br />
14
15 **Part 2:**<br />
16 **[Group 1 video]
17 (https://drive.google.com/file/d/1zmAk6lVv0VygGwITLWrSjZtySX
18 mFwt7B/view?usp=sharing)**<br />
19 **[Group 2 video]
20 (https://drive.google.com/file/d/1aqunP8Vc9hwKfLcwr0nZzIBru6
21 AJVbkC/view?usp=sharing)**<br />
22
23 # Homework
24
25 1. **Add README.md**
26
27 1. Create a branch with your task about git lesson
28 2. Add yourself and your mentor into README.md
29 3. Commit and push these changes
30 4. Create a pull request and set your mentor as a reviewer
```

- Man: open selected word ⌘ F6
- Go to Definition F12
- Go to References ⇧ F12
- Peek →
- Find All References ⌘ ⇧ F12
- Copilot →
- Rename Symbol ⇧ F6
- Change All Occurrences ⌘ F2
- Format Document ⌘ ⌘ L
- Format Document With... ⌘ ⌘ L
- Format Selection ⌘ ⌘ L
- Refactor... ^ T
- Source Action...
- Open Changes →
- Open on Remote (Web) →
- File Annotations →
- File History →
- Share →
- Stage Selected Ranges [⌘ K ⌘ S]
- Revert Selected Ranges [⌘ K ⌘ R]
- Cut ⌘ X
- Copy ⌘ C

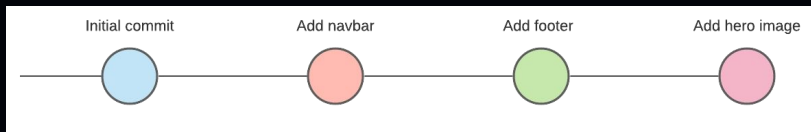
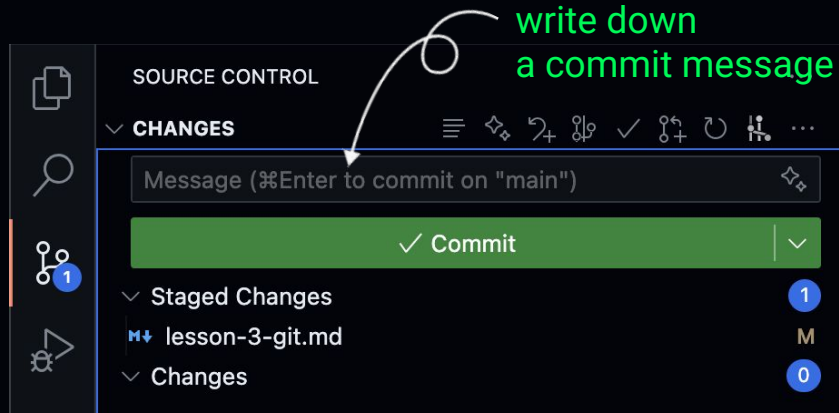
COMMITTS

THAT CREATES THE HISTORY



Commit

is set of changes to commit to git history



- **Use a descriptive commit message**
✓ good: "fix validation in contact form", ✗ bad: "fix", "sdf sdf"
- **Use the imperative mood**
✓ good: "fix validation", ✗ bad: "fixed validation"
- **Make each commit a logical unit**
don't commit more than described
- **Make atomic commits**
logical units, the smallest possible size
- **Strive to use no more than 60 symbols**
to be visible in GitHub UI



PUSH

publish your local commits

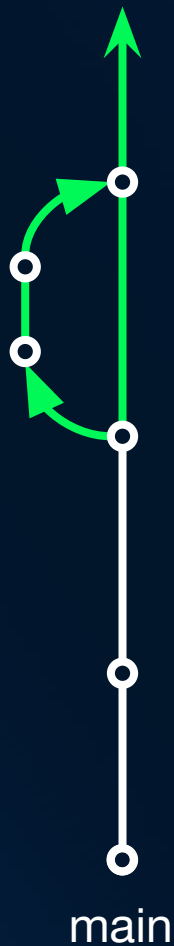


remote



Push

Push will publish your
local commits to the
remote server



local

PULL REQUEST

asking for merge your branch



Creating pull request

Using github.com

1. Open the repository on **GitHub** and select the **Pull Requests** tab.
2. Click the **New pull request** button.
3. A panel with the commits to be included in the pull request will appear.
Check branches names.
Click the **Create pull request** button.
4. Describe the changes made in your commits and click **Create pull request**.
5. Review the created pull request.

The screenshots illustrate the following steps:

- Step 1:** The GitHub repository page for 'sealeye / magic-vacation' is shown. The 'Pull requests' tab is selected in the top navigation bar.
- Step 2:** The 'New pull request' button is highlighted in the top right corner of the page.
- Step 3:** The 'Comparing changes' panel is shown. It displays the base repository (htmlcademy-animation/145...) and the head repository (sealeye/magic-vacation). The 'Create pull request' button is highlighted.
- Step 4:** The 'Open a pull request' panel is shown. The 'Update README.md' commit is selected. The 'Create pull request' button is highlighted.

The final screenshot shows the 'Update README.md #2' pull request page. It displays the commit details, the pull request title, and the 'Merge pull request' button.

MERGE

pulling a branch as a commit

Merge commit

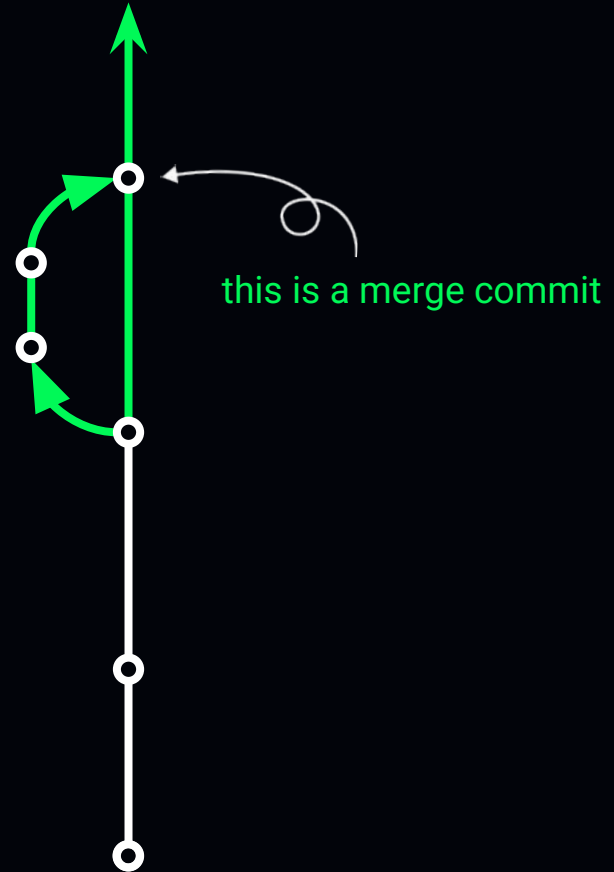
is a commit that pulls a branch into your current one

Usually looks like this:

```
Merge branch 'branch-name' into main'
```

```
COMMITS OF A BRANCH
```

```
See pull request LINK
```



Let's recap Git workflow for a task:

1. Create a new branch for a current lesson.
↓
2. Write some code.
↓
3. Commit the changes, specifying what was done in a commit message.
↓
4. Push the changes to the repository.
↓
5. Create a pull request.
↓
6. Change the status in Jira to "In review".
Include a link to the open pull request in the URL task field.
↓
7. If the task is reopened, repeat the process from step 2.
A new branch is not needed.
↓
8. Merge PR after approve
↓
9. Pull changes from remote master to local master

Usual Jira task workflow:



You Are

AWESOME

B Academy
RO



QUESTIONS?

Homework

Made two iterations of pull requests



Git lesson (add readme)

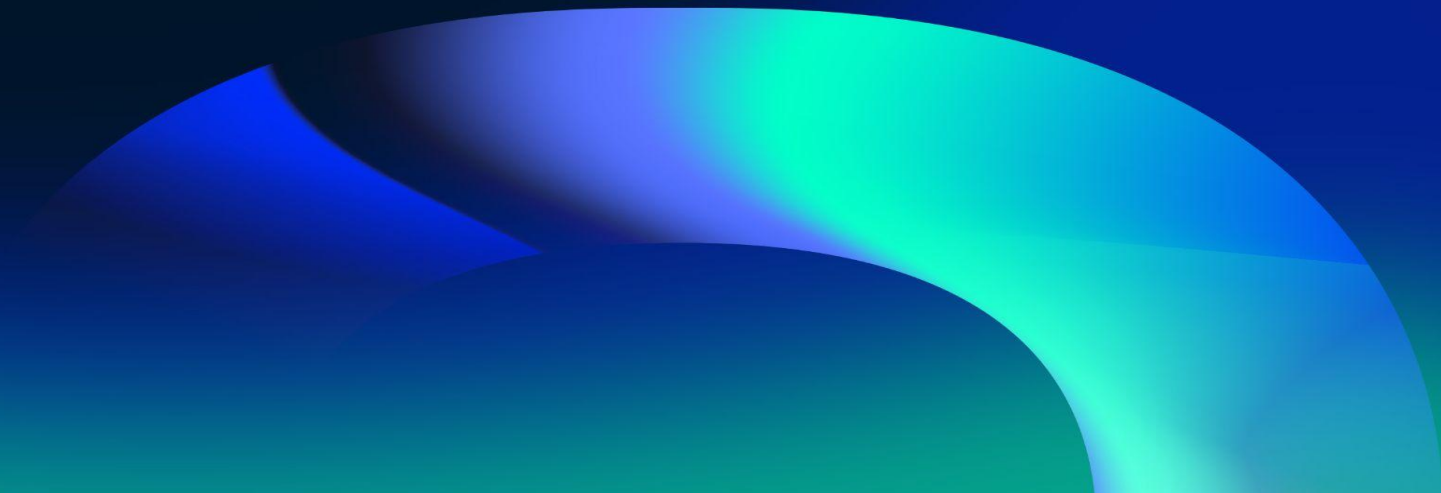
- 1.1. Create a branch with your task about git lesson
- 1.2. Add yourself and your mentor into README.md
- 1.3. Commit and push these changes
- 1.4. Create a pull request and set your mentor as a reviewer
- 1.5. Wait until your mentor will merge your branch into target branch

2. Save results of HTML lesson

- 2.1. Move back to target branch
- 2.2. Create another branch with your task about html lesson
- 2.3. Add your HTML files (index.html, blog.html)
- 2.4. Commit and push
- 2.5. Create a pull request and set your mentor as a reviewer
- 2.6. Wait until your mentor will merge your branch into target branch



Please fill out the feedback form
It's very important for us





THANK YOU!

Have a good evening!